

Guía de Grafos, ej 1 y 2: Implementación con Listas de Adyacencia

Repaso de teoría - Idea Básica

Sea m la cantidad de nodos del grafo dirigido (digrafo). (No es preciso conocer m de antemano).

Se define grafos implementados como lista de adyacencia del grafo G , a la implementación usando estructuras dinámicas autoreferenciadas con una estructura para los nodos y una para arcos, la implementación prevé además un puntero de acceso del tipo puntero a Nodo.

Definición de la Estructura En C++

```
struct nodo_grafo
{
    int id_nodo; // carga útil del nodo
    // nota1: resto de fx de asignación a nodo
    struct nodo_arco* lista_arco;
    struct nodo_grafo* link;
};

typedef struct nodo_grafo NGrafo;
// Nota 1: además cada nodo tiene un puntero de asignación a una estructura
// dinámica que contiene la información de nodos (fx de asignación a nodo).

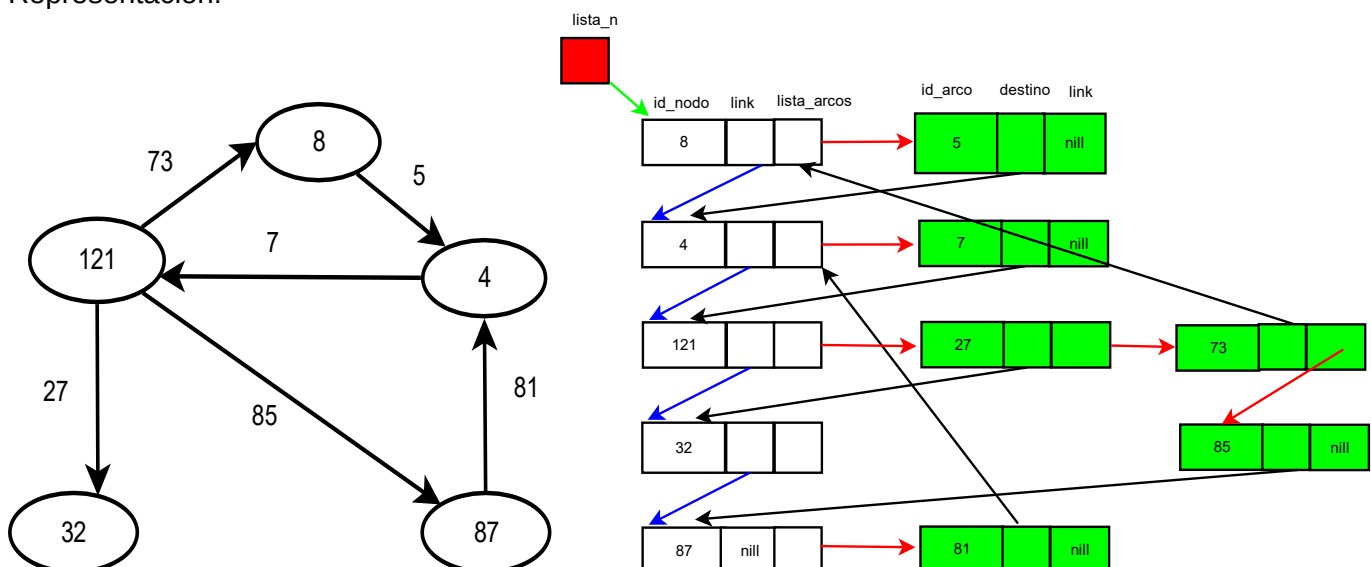
struct nodo_arco
{
    int id_arco; // carga útil del arco
    // nota2: resto de fx de asignación a arco
    struct nodo_grafo* destino;
    struct nodo_arco* link;

    typedef struct nodo_arco NArco;
    // Nota 2: además cada nodo tiene un puntero de asignación a una estructura
    // dinámica que contiene la información de arcos (fx de asignación al arco).
```

Punto de acceso:

```
NGrafo* lista_n = NULL;
```

Representación:



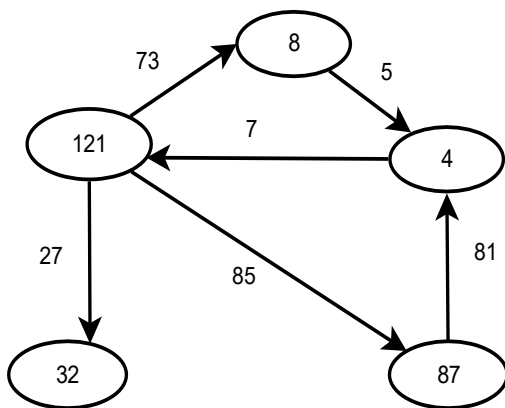
Características

- M puede ser desconocido con lo cual, a diferencia de la implementación con matriz de adyacencia es posible agregar y quitar nodos en cualquier momento.
- Optimizan el uso del almacenamiento.
- Es un poco más complejo las consultas y mantenimiento del grafo.

Análisis del ejercicio 1 y 2 de la Guía

Usar las funciones desarrolladas por la cátedra como base de esta práctica.

- 1) Dado el siguiente Grafo siguiente, genere el mismo con las funciones de referencia implementadas por la cátedra. Representar con opción 1 (Mostrar).



Grafo:

Nodo 87:

Arco 81 -> Nodo 4

Nodo 32:

Nodo 121:

Arco 73 -> Nodo 8

Arco 85 -> Nodo 87

Arco 27 -> Nodo 32

Nodo 4:

Arco 7 -> Nodo 121

Nodo 8:

Arco 5 -> Nodo 4

Solución: Usamos las funciones analizadas en clases anteriores.

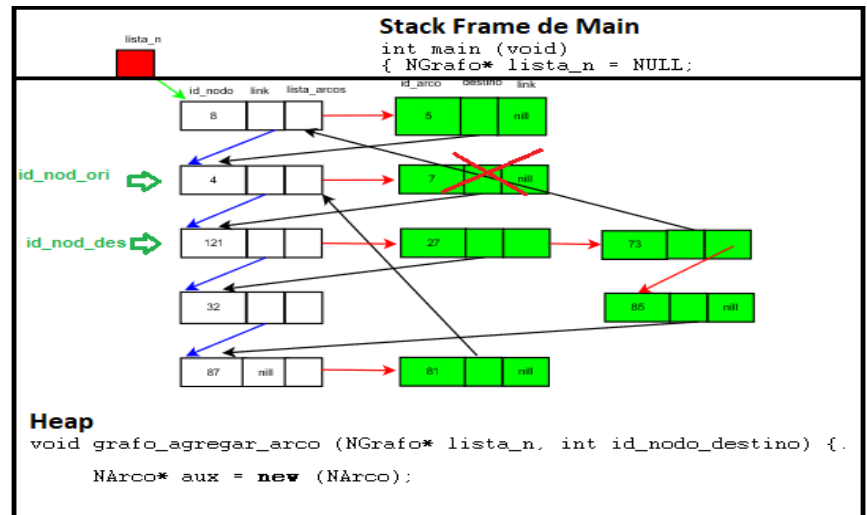
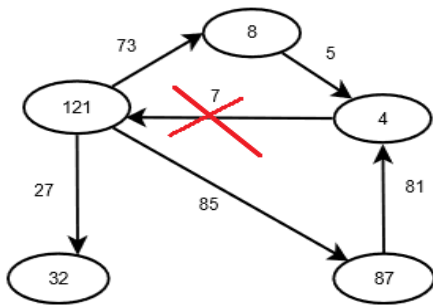
```
void menu_opcion5 (NGrafo* &lista_n)
{
    grafo_agregar_nodo (lista_n, 8);
    grafo_agregar_nodo (lista_n, 4);
    grafo_agregar_nodo (lista_n, 121);
    grafo_agregar_nodo (lista_n, 32);
    grafo_agregar_nodo (lista_n, 87);

    grafo_agregar_arco (lista_n, 5, 8, 4);
    grafo_agregar_arco (lista_n, 7, 4, 121);
    grafo_agregar_arco (lista_n, 73, 121, 8);
    grafo_agregar_arco (lista_n, 27, 121, 32);
    grafo_agregar_arco (lista_n, 81, 87, 4);

    cout << endl;
    cout << endl;
}
```

- 2) Realice un algoritmo que elimine un arco, para esto deberá solicitar el id de nodo origen y destino procediendo en consecuencia. Tratar las condiciones invalidas origen no existente y arco no existente.

Análisis:



Solución: Implementamos las ideas del análisis.

```
void grafo_eliminar_arco ( NGrafo* lista_n,
                          int id_nodo_origen,
                          int id_nodo_destino) {
    NGrafo* nodo_origen = lista_n;

    while ( nodo_origen != NULL &&
            nodo_origen->id_nodo != id_nodo_origen)
        nodo_origen = nodo_origen->link;

    // Verificamos si existe id_nodo_origen
    if (nodo_origen == NULL )
    {
        throw -1;        // Origen Inexistente
        return;
    }

    NArco* anterior    = NULL;
    NArco* actual      = nodo_origen->lista_arco;

    while (actual != NULL &&
            actual->destino->id_nodo != id_nodo_destino) {
        anterior = actual;
        actual = actual->link;
    }

    if (actual == NULL) {
        throw -2;        // Arco Inexistente
    } else {
        // actual != NULL hay nodo a borrar
        if (anterior == NULL && actual != NULL) {
            // quiere decir que es el 1er arco, hay que
            // actualizar el puntero del nodo
            nodo_origen->lista_arco = actual->link;
        } else {
            // es un arco intermedio o el ultimo
            anterior->link = actual->link;
        }
        delete actual;
    }
}
```